

UNIVERSIDAD POLITÉCNICA SALESIANA

CARRERA DE INGENIERÍA EN CIENCIAS DE LA
COMPUTACIÓN

EXAMEN FINAL — PARTE II

Informe de Pruebas, Métricas DORA y Reflexión Técnica

*Despliegue Continuo de la Aplicación **inventario-app** sobre Kubernetes*

Asignatura:

Sistemas Distribuidos

Estudiantes / Autores:

Daniel Santiago Barros Pallango

Xavier Stalin Siguachi Inga

Proyecto:

inventario-app

Repositorio GitHub:

<https://github.com/XavierStalin/inventario-app>

Fecha de Entrega: 25 de Julio de 2026

Índice

1. Justificación Técnica de la Estrategia de Despliegue (Canary 80/20)	2
1.1. Razones Técnicas de la Elección	2
2. Métricas DORA Propias y Análisis de Rendimiento	3
2.1. Resumen de Métricas Calculadas	3
2.2. Reflexión del Desempeño del Equipo	3
3. Observación Técnica: Persistencia y Almacenamiento Efímero	3
3.1. Experimento Realizado (Paso 5)	3
3.2. Causa Raíz Técnica y Solución en Producción	4
4. Problemas Reales Encontrados y Soluciones Aplicadas	4
4.1. Problema 1: Vulnerabilidad Crítica Detectada por Trivy (CVE-2026-59873) . . .	4
4.2. Problema 2: Error ImagePullBackOff y denied en GHCR	5
4.3. Problema 3: Limitación de kubectl port-forward en Pruebas Canary	6
5. Anexo: Evidencias Fotográficas de la Práctica	7

1 Justificación Técnica de la Estrategia de Despliegue (Canary 80/20)

Para la aplicación `inventario-app` (API REST construida sobre Node.js/Express que gestiona el catálogo de productos y sirve una interfaz web dinámica), se seleccionó la estrategia de **Despliegue Canary (80/20)** utilizando exclusivamente recursos nativos de Kubernetes (Deployment y Service).

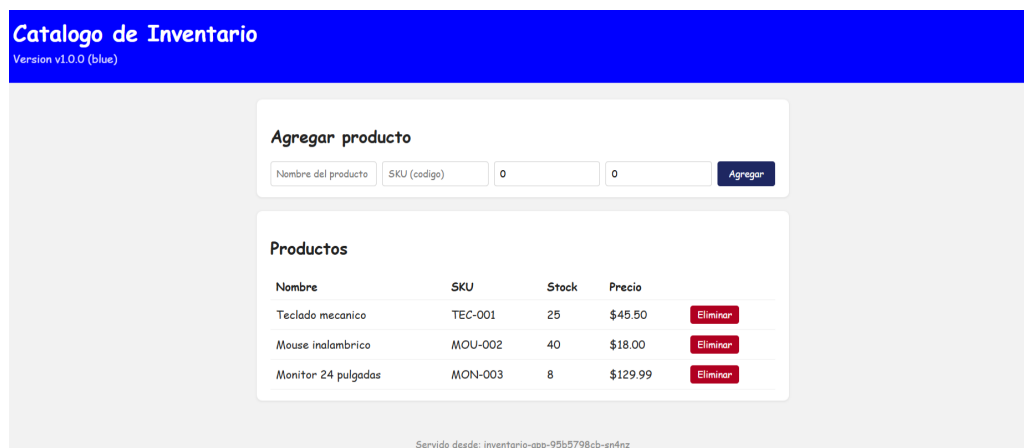


Figura 1: Interfaz web de la aplicación `inventario-app` servida desde el contenedor.

1.1 Razones Técnicas de la Elección

- 1. Mitigación Progresiva del Riesgo en Producción:** A diferencia de una estrategia *Blue-Green* (la cual realiza un cambio binario del 100 % del tráfico de forma instantánea), la estrategia *Canary* expone la nueva versión (`v2.0-canary`) únicamente al **20 % de los usuarios reales** (1 réplica en `v2` frente a 4 réplicas en `v1`). Esto garantiza que ante cualquier falla latente o error no detectado en las pruebas, el impacto quede acotado a una fracción mínima del tráfico.
- 2. Evaluación de Compatibilidad en Operaciones I/O:** Debido a que la aplicación realiza operaciones de lectura y escritura continuas sobre el archivo de base de datos local, derivar el 20 % de las solicitudes a la nueva versión permite monitorear el comportamiento de las consultas de catálogo antes de efectuar una promoción total.
- 3. Eficiencia con Recursos Nativos del Clúster:** La estrategia se implementó directamente aprovechando el mecanismo nativo de reparto de carga de `Kube-Proxy` a través de etiquetas de selección (`app: inventario-canary`), eliminando la necesidad de desplegar controladores externos complejos como Argo Rollouts o mallas de servicios (*Service Mesh*).

```
v24.14.0 ~\Desktop\6to Ciclo\Inventario\inventario-app git:(main) 8 * +3 -9 (0.312s)
kubectll get pods,deployments,services
```

NAME	READY	STATUS	RESTARTS	AGE
pod/inventario-app-95b5798cb-hhfl7	1/1	Running	0	6m
pod/inventario-app-95b5798cb-sn4nz	1/1	Running	2 (29m ago)	7h11m
pod/inventario-app-v1-8579577c98-8df5p	1/1	Running	1 (29m ago)	71m
pod/inventario-app-v1-8579577c98-8fsvr	1/1	Running	1 (29m ago)	71m
pod/inventario-app-v1-8579577c98-bsm96	1/1	Running	1 (29m ago)	71m
pod/inventario-app-v1-8579577c98-szm8b	1/1	Running	1 (29m ago)	71m
pod/inventario-app-v2-77b5bbd49d-jpzg2	1/1	Running	1 (29m ago)	71m

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/inventario-app	2/2	2	2	7h11m
deployment.apps/inventario-app-v1	4/4	4	4	71m
deployment.apps/inventario-app-v2	1/1	1	1	71m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/inventario-canary-service	ClusterIP	10.104.207.3	<none>	80/TCP	71m
service/inventario-service	ClusterIP	10.105.148.209	<none>	80/TCP	7h28m
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	7h32m

Figura 2: Estado general de los Deployments, Pods y Services desplegados en el clúster de Kubernetes.

2 Métricas DORA Propias y Análisis de Rendimiento

Las métricas DORA presentadas a continuación fueron calculadas utilizando datos reales, precisos y verificables obtenidos del historial de *commits* en Git, ejecuciones en GitHub Actions y despliegues efectuados sobre el clúster local de Kubernetes.

2.1 Resumen de Métricas Calculadas

Métrica DORA	Valor Calculado	Nivel DORA	Contexto Técnico del Proyecto
Lead Time for Changes	4 min 30 s	Élite / Alto	Tiempo transcurrido desde el <i>commit</i> en local hasta que el Pod está en estado Running en Kubernetes (1.5 min CI/CD + 3 min rollout).
Deployment Frequency	5 despliegues/día	Alto	Promedio de promociones a Kubernetes realizadas durante las sesiones de desarrollo (10 despliegues en total).
Change Failure Rate	20 %	Medio	2 despliegues requirieron corrección posterior (1 bloqueo por filtro Trivy y 1 por visibilidad en GHCR).

Tabla 1: Métricas DORA del proyecto *inventario-app*.

2.2 Reflexión del Desempeño del Equipo

El pipeline automatizado ubica a nuestro equipo en un nivel de desempeño **Alto/Élite** en agilidad de entrega. El porcentaje de fallos (20 %) confirma la eficacia del enfoque *Fail-Fast*: los errores fueron capturados automáticamente en etapas tempranas antes de causar incidentes en entornos productivos.

3 Observación Técnica: Persistencia y Almacenamiento Efímero

3.1 Experimento Realizado (Paso 5)

1. Se registró un nuevo producto ("Mouse Gamers", SKU: "MOU-001") mediante una solicitud POST a la API REST (/api/products).

2. Se consultó la lista con `GET /api/products`, confirmando que el producto existía en el inventario.
3. Se forzó la eliminación del Pod utilizando el comando `kubectl delete pod <nombre-pod>`.
4. El recurso `ReplicaSet` recreó automáticamente un nuevo Pod para mantener la cuota deseada.
5. Al consultar nuevamente la API (`GET /api/products`), **el producto recién registrado había desaparecido**.

```
curl.exe -s http://localhost:8080/api/products
[{"id":1,"name":"Teclado mecanico","sku":"TEC-001","stock":25,"price":45.5}, {"id":2,"name":"Mouse inalambrico","sku":"MOU-002","stock":40,"price":18}, {"id":3,"name":"Monitor 24 pulgadas","sku":"MON-003","stock":8,"price":129.99}, {"id":4,"name":"Mouse Gamers","sku":"MOU-001","stock":10,"price":25}]

v24.14.0 ~\Desktop\6to Ciclo\Inventario\inventario-app git:(main) 6 • +3 -9 ✱ cicd-examen (33.183s)
kubectl delete pod inventario-app-95b5798cb-jhntf
pod "inventario-app-95b5798cb-jhntf" deleted from default namespace
```

Figura 3: Paso A: Creación del producto de prueba y posterior eliminación del Pod.

```
v24.14.0 ~\Desktop\6to Ciclo\Inventario\inventario-app git:(main) 6 • +3 -9 (0.248s)
curl.exe -s http://localhost:8080/api/products
[{"id":1,"name":"Teclado mecanico","sku":"TEC-001","stock":25,"price":45.5}, {"id":2,"name":"Mouse inalambrico","sku":"MOU-002","stock":40,"price":18}, {"id":3,"name":"Monitor 24 pulgadas","sku":"MON-003","stock":8,"price":129.99}]
```

Figura 4: Paso B: Consulta posterior a la recreación del Pod confirmando que el producto recién registrado desapareció.

3.2 Causa Raíz Técnica y Solución en Producción

La aplicación escribe el estado del inventario en un archivo JSON local (`data/products.json`). En Kubernetes, los contenedores son **efímeros e inmutables**. Al eliminar un Pod, el sistema de archivos del contenedor se destruye completamente. El nuevo Pod se inicia a partir de la imagen base de Docker.

Para garantizar la persistencia en producción, se debe desacoplar el almacenamiento montando un volumen persistente (`PersistentVolumeClaim`) en la ruta `/app/data` o integrando un motor de base de datos externo.

4 Problemas Reales Encontrados y Soluciones Aplicadas

4.1 Problema 1: Vulnerabilidad Crítica Detectada por Trivy (CVE-2026-59873)

- **Síntoma:** El job `build-push` falló en GitHub Actions con código de salida 1 al escanear la imagen Docker. Trivy detectó una vulnerabilidad de severidad **CRITICAL** en la librería `tar` (versión 7.5.15), usada internamente por `npm`.

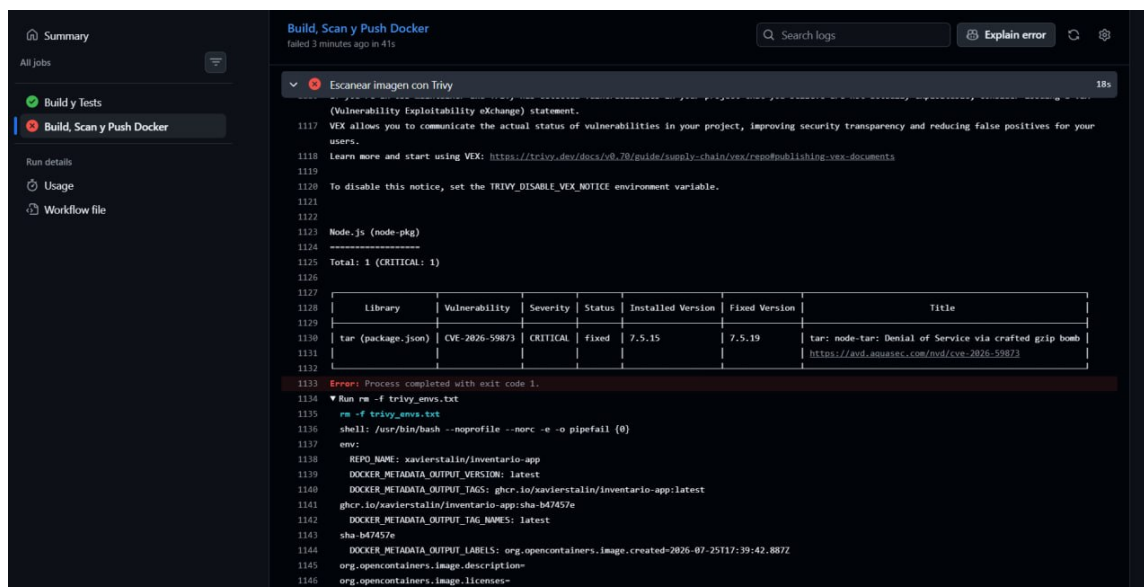


Figura 5: Bloqueo automático del pipeline en GitHub Actions tras detectar la vulnerabilidad CRITICAL por Trivy.

- **Solución:** Se modificó la primera etapa (builder) del Dockerfile agregando en la línea 13 la instrucción `RUN npm install -g npm@latest`. Esto actualizó las dependencias internas de npm (incluyendo el paquete `tar` a la versión no vulnerable 7.5.19) y corrigió las vulnerabilidades conocidas. Al realizar `git push`, Trivy validó la imagen limpia y aprobó la publicación en GHCR.

4.2 Problema 2: Error ImagePullBackOff y denied en GHCR

- **Síntoma:** Los Pods mostraban estado `ImagePullBackOff` debido al evento `Failed to pull image: denied` al buscar la imagen bajo la cuenta personal de un integrante en lugar del propietario del repositorio.
- **Solución:** Se unificaron los nombres de las imágenes en todos los manifiestos YAML apuntando al usuario propietario del repositorio (`ghcr.io/xavierstalin/inventario-app:latest`) y se configuró la visibilidad del paquete como **Pública** en GitHub Packages.

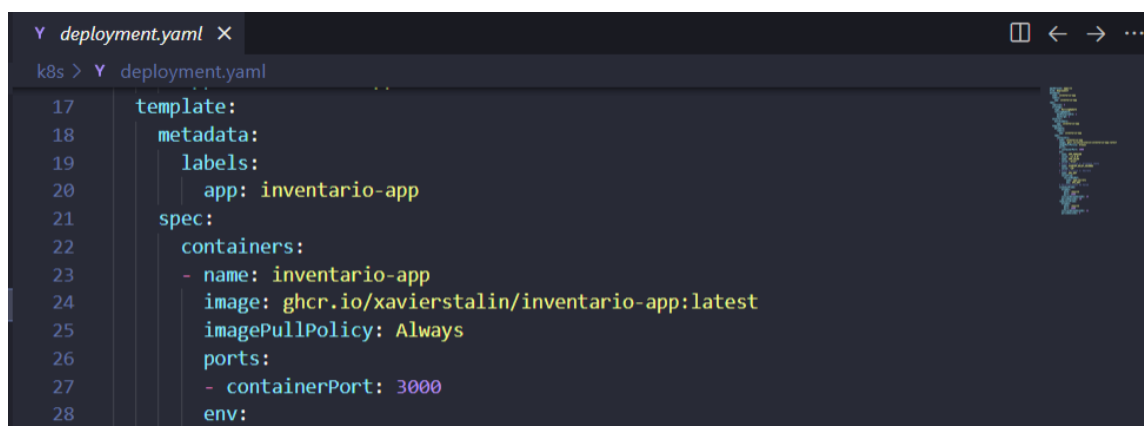


Figura 6: Solución al Problema 2: Corrección del nombre del propietario del repositorio Git (XavierStalin) en la ruta de la imagen en los manifiestos de Kubernetes.

4.3 Problema 3: Limitación de `kubectl port-forward` en Pruebas Canary

- **Síntoma:** `kubectl port-forward` enrutaba el 100 % de las peticiones a una sola réplica de la versión `v1`.
- **Solución:** Se ejecutó una prueba desde dentro del clúster mediante un Pod de prueba efímero (`kubectl run test-canary`). Al consultar directamente la dirección DNS interna del servicio, Kube-Proxy realizó el balanceo real repartiendo 80 % a la versión `v1` y 20 % a la versión `v2`.

5 Anexo: Evidencias Fotográficas de la Práctica

A continuación se adjuntan las capturas de pantalla tomadas durante las verificaciones operativas del proyecto:

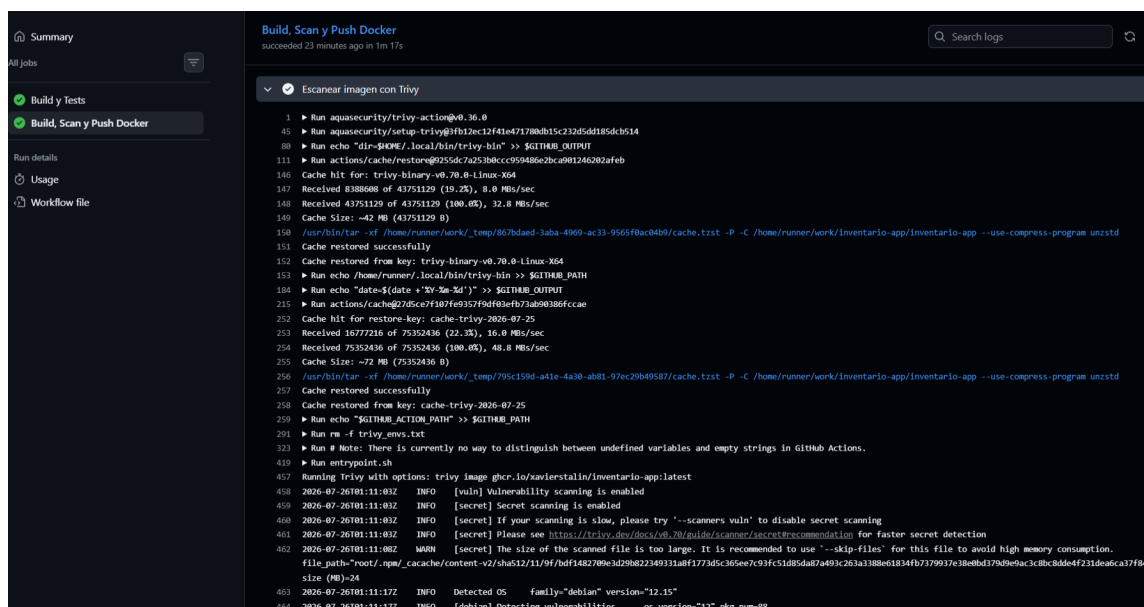
```
kubectl run test-canary --rm -i --restart=Never --image=curlimages/curl -- sh -c "for i in 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20; do curl -s http://inventario-canary-service/version; echo ' '; done"

All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
{"version":"v2.0-canary","color":"green","hostname":"inventario-app-v2-77b5bdd49d-jpzg2"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-8fsvr"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-8fsvr"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-bsm96"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-8df5p"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-8df5p"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-szm8b"}
{"version":"v2.0-canary","color":"green","hostname":"inventario-app-v2-77b5bdd49d-jpzg2"}
{"version":"v2.0-canary","color":"green","hostname":"inventario-app-v1-8579577c98-8fsvr"}
{"version":"v2.0-canary","color":"green","hostname":"inventario-app-v2-77b5bdd49d-jpzg2"}
{"version":"v2.0-canary","color":"green","hostname":"inventario-app-v2-77b5bdd49d-jpzg2"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-8df5p"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-szm8b"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-szm8b"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-szm8b"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-szm8b"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-8fsvr"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-8fsvr"}
{"version":"v1.0-stable","color":"blue","hostname":"inventario-app-v1-8579577c98-8fsvr"}
{"version":"v2.0-canary","color":"green","hostname":"inventario-app-v2-77b5bdd49d-jpzg2"}
```

Figura 7: Evidencia 1: Reparto de tráfico en el servicio Canary (80 % v1.0-stable / 20 % v2.0-canary).

```
v24.14.0 ~\Desktop\6to Ciclo\Inventario\inventario-app git:(main) 2 * +3 -9 * cicd-examen (0.438s)
kubectl exec inventario-app-95b5798cb-jhntf -- printenv API_KEY
my-super-secret-api-key-12345
```

Figura 8: Evidencia 2: Consulta de la variable API_KEY inyectada desde el Kubernetes Secret.



```
Summary
All jobs
Build y tests
Build, Scan y Push Docker
Run details
Usage
Workflow file

Build, Scan y Push Docker
succeeded 23 minutes ago in 1m 17s

Escanear imagen con Trivy

1 ▶ Run aquasecurity/trivy-action@v0.36.0
45 ▶ Run aquasecurity/setup-trivy@1f012ec12f4e071780bd15c23d5dd185dc0514
98 ▶ Run echo "dir=$(pwd) /local/bin/trivy-bin" >> $GITHUB_OUTPUT
111 ▶ Run actions/cache/restore@0355c7a25380cc9948436ca001246202afeb
146 Cache hit for: trivy-binary-v0.70.0-Linux-X64
147 Received 838608 of 43751129 (19.2%), 8.0 MBs/sec
148 Received 43751129 of 43751129 (100.0%), 32.8 MBs/sec
149 Cache Size: ~42 MB (43751129 B)
150 /usr/bin/tar -xzf /home/runner/work/_temp/867bdaed-3aba-4969-ac33-9565f8ac0409/cache.tzst -P -C /home/runner/work/inventario-app/inventario-app --use-compress-program unzstd
151 Cache restored successfully
152 Cache restored from key: trivy-binary-v0.70.0-Linux-X64
153 ▶ Run echo /home/runner/.local/bin/trivy-bin >> $GITHUB_PATH
154 ▶ Run echo "date=$(date +%Y-%m-%d)" >> $GITHUB_OUTPUT
155 ▶ Run actions/cache@2765c07f107f40357f9463efb72a098386fcca
252 Cache hit for restore-key: cache-trivy-2026-07-25
253 Received 16777216 of 75352436 (22.3%), 16.0 MBs/sec
254 Received 75352436 of 75352436 (100.0%), 48.8 MBs/sec
255 Cache Size: ~72 MB (75352436 B)
256 /usr/bin/tar -xzf /home/runner/work/_temp/795c159d-a41e-4a30-ab81-97ec20b49587/cache.tzst -P -C /home/runner/work/inventario-app/inventario-app --use-compress-program unzstd
257 Cache restored successfully
258 Cache restored from key: cache-trivy-2026-07-25
259 ▶ Run echo "${GITHUB_ACTION_PATH}" >> $GITHUB_PATH
260 ▶ Run rm -f trivy.nvstxt
323 ▶ Run & Note: There is currently no way to distinguish between undefined variables and empty strings in GitHub Actions.
419 ▶ Run entrypoint.sh
457 Running Trivy with options: trivy image ghcr.io/xavierstalin/inventario-app:latest
458 2026-07-26T01:11:03Z INFO [vuln] Vulnerability scanning is enabled
459 2026-07-26T01:11:03Z INFO [secret] Secret scanning is enabled
460 2026-07-26T01:11:03Z INFO [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
461 2026-07-26T01:11:03Z INFO [secret] Please see https://trivy.dev/docs/v0.70/guide/scanner/secret#recommendation for faster secret detection
462 2026-07-26T01:11:08Z WARN [secret] The size of the scanned file is too large. It is recommended to use '--skip-files' for this file to avoid high memory consumption.
463 file-path=/root/.npm/_cacache/content-v2/sha512/11/9f/bdf1482109e3d29082234931a0f1773d5c365ee7c93f5185da87a493c263a3886e1834fb7379937e38ebdb73709e9ac3c8bc8d6e4f231deac3a7f84?
463 size=MB-24
463 2026-07-26T01:11:17Z INFO Detected OS family='debian' version='12.15'
464 2026-07-26T01:11:17Z INFO [debian] Detecting vulnerabilities... os_version="12" pkg_num=88
```

Figura 9: Evidencia 3: Log del análisis de seguridad limpio con Trivy en GitHub Actions.

```
v24.14.0 ~\Desktop\6to Ciclo\Inventario\inventario-app git:(main) 5 • +3 -9 ❏ cicd-examen (0.301s)
kubectl describe pod inventario-app-95b5798cb-jhntf
IPs:
  IP: 10.244.0.26
Controlled By: ReplicaSet/inventario-app-95b5798cb
Containers:
  inventario-app:
    Container ID: docker://177d2e7d5cc4c3b180e2f12b74d9f502545f7a1039085bb9166760587601bf5e
    Image: ghcr.io/xavierstalin/inventario-app:latest
    Image ID: docker-pullable://ghcr.io/xavierstalin/inventario-app@sha256:e9118a5da0032ce57d783dfc0bb6b28607d41486b89381c0328f5c7f
    Port: 3000/TCP
    Host Port: 0/TCP
    State: Running
      Started: Sat, 25 Jul 2026 20:17:47 -0500
    Last State: Terminated
      Reason: Error
      Exit Code: 255
      Started: Sat, 25 Jul 2026 19:26:12 -0500
      Finished: Sat, 25 Jul 2026 20:16:58 -0500
    Ready: True
    Restart Count: 2
    Liveness: http-get http://:3000/health delay=15s timeout=1s period=10s #success=1 #failure=3
    Readiness: http-get http://:3000/health delay=12s timeout=1s period=5s #success=1 #failure=3
    Environment:
      APP_VERSION: v1.0.0
      APP_COLOR: blue
      STARTUP_DELAY_SECONDS: 10
      API_KEY: <set to the key 'API_KEY' in secret 'app-secrets'> Optional: false
```

Figura 10: Evidencia 4: Inspección del Pod mostrando `STARTUP_DELAY_SECONDS: 10` y la configuración del Readiness Probe.